

NodeJS Interview Questions and Answers

Last Updated : 06 Mar, 2025

NodeJS is one of the most popular runtime environments, known for its efficiency, scalability, and ability to handle asynchronous operations. It is built on **Chrome's V8 JavaScript engine** for executing JavaScript code outside of a browser. It is extensively used by top companies such as LinkedIn, Netflix, Walmart, Uber, PayPal, NASA, and many more because of its robust features and performance.

In this guide, we will provide **50+ NodeJS Interview Questions and Answers 2025** designed for freshers and experienced professionals with 3, 5, and 8 years of experience. Here, we cover everything, including core NodeJS concepts, asynchronous programming, event-driven architecture, error handling, design patterns, NodeJS modules, and more, that will surely help you to crack NodeJS interviews.

Table of Content

- NodeJS Interview Questions for Freshers
- NodeJS Intermediate Interview Questions
- NodeJS Interview Questions for Experienced

NodeJS Interview Questions for Freshers

In this section, we will discuss basic NodeJS questions asked in the interview suitable for the freshers.

1. What is NodeJS?

NodeJS is an open-source, cross-platform JavaScript runtime environment engine used for executing JavaScript code outside the

browser. It is built on Google Chrome's V8 JavaScript engine. Some of

- Single-threaded
- Non-Blocking, Asynchronous I/O
- Cross-platform
- Fast Execution (V8 Engine)
- Real-Time Data Handling

2. What is NPM?

NPM stands for the Node Package Manager. It is the package manager for the NodeJS environment. It is used to install, share, and manage dependencies (libraries, tools, or packages) in JavaScript applications. Below are the following key points about the NPM:

- NPM uses a package.json file in NodeJS projects to track project dependencies, versions, scripts, and metadata like the project's name and version.
- NPM is accessed by a command-line interface (CLI). Common commands include npm install to install packages, npm update to update them, and npm uninstall to remove them.

3. Why NodeJS is single-threaded?

NodeJS is single-threaded because it's based on the asynchronous, non-blocking nature of JavaScript. This design makes it simpler to develop and maintain, and it allows NodeJS to handle many concurrent requests efficiently.

4. What kind of API function is supported by NodeJS?

There are two types of API functions supported by NodeJS:

- **Synchronous:** These API functions are used for blocking code.
- **Asynchronous:** These API functions are used for non-blocking code.

5. What is the difference between Synchronous and Asynchronous functions?

Here we have difference table between Synchronous and Asynchronous functions

Feature	Synchronous Functions	Asynchronous Functions
Execution Blocking	Blocks the execution until the task completes.	Does not block the execution; allows other tasks to proceed concurrently.
Waiting for Completion	Executes tasks sequentially; each task must complete before the next one starts.	Initiates tasks and proceeds with other operations while waiting for completion.
Return Value	Returns the result immediately after completion.	Typically returns a promise, callback, or uses event handling to handle the result upon completion.
Error Handling	Errors can be easily caught with try-catch blocks.	Error handling is more complex and often involves callbacks, promises, or async/await syntax.
Usage Scenario	Suitable for simple, sequential tasks with predictable execution flow.	Ideal for I/O-bound operations, network requests, and tasks requiring parallel processing.

6. What is a module in NodeJS?

In NodeJS Application, a Module can be considered as a block of code that provide a simple or complex functionality that can communicate

with external application. Modules can be organized in a single file or a collection of multiple files/folders. They are useful because of their reusability and ability to reduce the complexity of code into smaller pieces. Examples of modules are. http, fs, os, path, etc.

A module in Node.js is a reusable piece of code that can be imported and exported between different files

7. What is the purpose of the 'require' keyword in NodeJS?

The require keyword in NodeJS is used to include and import modules (external or built-in) into a NodeJS application.

Example

```
const http = require('http')   //imports the HTTP module to
create a server.
```

8. What is middleware?

Middleware is the function that works between the request and the response cycle. Middleware gets executed after the server receives the request and before the controller sends the response.

9. How does NodeJS handle concurrency even after being single-threaded?

NodeJS handles concurrency by using asynchronous, non-blocking operations. Instead of waiting for one task to complete before starting the next, it can initiate multiple tasks and continue processing while waiting for them to finish, all within a single thread.

10. What is control flow in NodeJS?

Control flow in NodeJS refers to the sequence in which statements and functions are executed. It manages the order of execution, handling asynchronous operations, callbacks, and error handling to ensure smooth program flow.

11. What do you mean by event loop in NodeJS?

The event loop in NodeJS is a mechanism that allows it to handle multiple asynchronous tasks concurrently within a single thread. It continuously listens for events and executes associated callback functions.

12. What is the order in which control flow statements get executed?

The order in which the statements are executed is as follows:

- Execution and queue handling
- Collection of data and storing it
- Handling concurrency
- Executing the next lines of code

13. What are the main disadvantages of NodeJS?

Here are some main disadvantages of NodeJS listed below:

- **Single-threaded nature:** May not fully utilize multi-core CPUs, limiting performance.
- **NoSQL preference:** Relational databases like MySQL aren't commonly used.
- **Rapid API changes:** Frequent updates can introduce instability and compatibility issues.

14. What is REPL in NodeJS?

REPL in NodeJS stands for Read, Evaluate, Print, and Loop. It is a computer environment similar to the shell which is useful for writing and debugging code as it executes the code in on go.

- **Read:** It reads the input provided by the user (JavaScript expressions or commands).
- **Eval:** It evaluates the input (executes the code).
- **Print**: It prints the result of the evaluation to the console.
- **Loop**: It loops back, allowing you to enter more code and get immediate results.

15. How to import a module in NodeJS?

We use the require module to import the External libraries in NodeJS. The result returned by require() is stored in a variable which is used to invoke the functions using the dot notation.

16. What is the difference between NodeJS and Angular?

Feature	NodeJS	Angular
Type	Server-side runtime environment	Front-end framework
Language	JavaScript (or TypeScript)	TypeScript (primarily, but supports JavaScript)
Execution	Runs on the server to handle requests and responses.	Runs on the client side (browser) to build user interfaces.
Performance	Highly efficient for I/O operations due to its non-blocking nature.	Performance-focused with optimization for large-scale single-page applications.
Usage	Primarily used for server-side JavaScript execution and backend services.	Used for building interactive, dynamic, and responsive front-end applications.

17. What is package.json in NodeJS?

package.json in NodeJS is a metadata file that contains project-specific information such as dependencies, scripts, version, author details, and other configuration settings required for managing and building the project.

Example:

```
{
    "name": "app",
    "version": "1.0.0",
    "main": "index.js",
    "scripts": {
        "test": "echo \"Error: no test specified\" && exit 1"
    },
    "keywords": [],
    "author": "",
    "license": "ISC",
    "description": "",
    "dependencies": {
        "express": "^4.21.2"
    }
}
```

18. How to create the simple HTTP server in NodeJS?

You can create a simple HTTP server in NodeJS using the built-in http module:

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello, World!');
});
server.listen(3000, () => {
  console.log('Server is running at http://localhost:3000/');
});
```

Output:

```
node app.js
```



NodeJS

19. What is the most commonly used libraries in NodeJS?

There are the two most commonly used libraries in NodeJs:

- **ExpressJS:** ExpressJS is a minimal and flexible web application framework for building robust APIs and web apps. It simplifies routing, middleware handling, and request/response management.
- **Mongoose:** An Object Data Modeling (ODM) library for MongoDB and NodeJS, it helps in managing data relationships, schema validation, and business logic.

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It provides a structured way to interact with MongoDB by defining schemas and models..ODM (Object Data Modeling) is a technique that maps objects in code to documents in a NoSQL database like MongoDB.

20. What are promises in NodeJS?

A promise is basically an advancement of callbacks in NodeJS. In other words, a promise is a JavaScript object which is used to handle all the asynchronous data operations. While developing an application you may encounter that you are using a lot of nested callback functions which causes a problem of callback hell. Promises solve this problem of callback hell.

NodeJS Intermediate Interview Questions

In this set we will be looking at intermediate Node Interview Question for candidates with over 2 years of experience.

21. What is event-driven programming in NodeJS?

Event-driven programming is used to synchronize the occurrence of multiple events and to make the program as simple as possible. The

basic components of an Event-Driven Program are:

- A callback function (called an event handler) is called when an event is triggered.
- An event loop that listens for event triggers and calls the corresponding event handler for that event.

22. What is a a buffer in NodeJS?

The Buffer class in NodeJS is used to perform operations on raw binary data. Generally, Buffer refers to the particular memory location in memory. Buffer and array have some similarities, but the difference is array can be any type, and it can be resizable. Buffers only deal with binary data, and it can not be resizable. Each integer in a buffer represents a byte. console.log() function is used to print the Buffer instance.

23. What are streams in NodeJS?

In NodeJS, streams are a powerful way to handle data in chunks rather than loading the entire data into memory. Streams allow for the efficient processing of large volumes of data, especially in situations where the data size is too large to fit into memory all at once.

There are the four types of the Streams:

- **Readable Streams:** These streams allow you to read data. For example, reading data from a file or receiving HTTP request data.
  Example:

```
fs.createReadStream() or http.IncomingMessage.
```

- **Writable Streams:** These streams allow you to write data. For example, writing data to a file or sending HTTP response data.
  Example:

```
fs.createWriteStream() or http.ServerResponse.
```

- **Duplex Streams:** These are both readable and writable. You can both read and write data using the same stream. **Example**: A TCP socket.

- **Transform Streams:** These are a type of duplex stream where the data is transformed as it is read and written. **Example:** A zlib stream to compress or decompress data.

24. Explain crypto module in NodeJS

The crypto module is used for encrypting, decrypting, or hashing any type of data. This encryption and decryption basically help to secure and add a layer of authentication to the data. The main use case of the crypto module is to convert the plain readable text to an encrypted format and decrypt it when required.

25. What is callback hell?

Callback hell is an issue caused due to a nested callback. This causes the code to look like a pyramid and makes it unable to read To overcome this situation we use promises.

26. Explain the use of timers module in NodeJS

The Timers module in NodeJS contains various functions that allow us to execute a block of code or a function after a set period of time. The Timers module is global, we do not need to use require() to import it.

It has the following methods:

- setTimeout() method
- setImmediate() method
- setInterval() method

27. Difference between setImmediate() and process.nextTick() methods

Feature	setImmediate()	process.nextTick()
Execution Timing	Executes the callback after the current event loop cycle, but before the I/O tasks.	Executes the callback immediately after the current operation, before any I/O or timers.
Priority	Executes after I/O events and before timers.	Executes before any I/O events or timers.
Stack Overflow Risk	Less likely to cause a stack overflow because it is queued after the current event loop phase.	Can cause a stack overflow if used excessively because it executes before I/O or other operations, potentially blocking the event loop.
Use Case	Used when you want to execute a callback after the event loop is done processing the current phase but before the next one starts.	Used to schedule a callback to be executed before any I/O events or timers in the current phase.
Example	setImmediate(() => { console.log('Immediate'); });	process.nextTick(() => { console.log('Next Tick'); });

28. What are the different types of HTTP requests?

The different types of the HTTP requests are mentioned below:

- **GET:** Retrieve data.
- **POST:** Create new resource.
- **PUT**: Update an entire resource.
- **PATCH**: Partially update a resource.
- **DELETE:** Remove a resource.

29. What is the difference between spawn() and fork() method?

Feature	spawn()	fork()
Purpose	Used to launch a new process with a given command.	Used specifically for spawning new NodeJS processes.
Return Type	Returns a ChildProcess object that allows interaction with the spawned process.	Returns a ChildProcess object with built-in support for IPC.
IPC Support	No built-in support for IPC (requires additional setup).	Built-in support for IPC, making it easier to communicate between the parent and child processes.
Use Case	Running external commands or executing scripts (e.g., shell scripts, commands).	Running NodeJS scripts in child processes that can send and receive messages from the parent process.
Example Command	spawn('ls', ['-lh', '/usr'])	fork('child.js')

30. Explain the use of passport module in NodeJS

The passport module is used for adding authentication features to our website or web app. It implements authentication measure which helps to perform sign-in operations.

31. What is a a fork in NodeJS?

Fork is a method in NodeJS that is used to create child processes. It helps to handle the increasing workload. It creates a new instance of the engine which enables multiple processes to run the code.

32. What are the three methods to avoid callback hell?

The three methods to avoid callback hell are:

- Using async/await()
- Using promises
- Using generators

33. What is a .body-parser in NodeJS?

Body-parser is the NodeJS body-parsing middleware. It is responsible for parsing the incoming request bodies in a middleware before you handle it. It is an NPM module that processes data sent in HTTP requests.

34. What is CORS in NodeJS?

The word CORS stands for “Cross-Origin Resource Sharing”. Cross-Origin Resource Sharing is an HTTP-header based mechanism implemented by the browser which allows a server or an API to indicate any origins (different in terms of protocol, hostname, or port) other than its origin from which the unknown origin gets permission to access and load resources. The cors package available in the npm registry is used to tackle CORS errors in a NodeJS application.

35. Explain the tls module in NodeJS..

The tls module provides an implementation of the Transport Layer Security (TLS) and Secure Socket Layer (SSL) protocols that are built on top of OpenSSL. It helps to establish a secure connection on the network.

NodeJS Interview Questions for Experienced

In this set we will be covering Node interview question for experienced developers with over 5 years of experience.

36. What is a cluster in NodeJS?

Due to a single thread in NodeJS, it handles memory more efficiently because there are no multiple threads due to which no thread management is needed. Now, to handle workload efficiently and to take advantage of computer multi-core systems, cluster modules are created that provide us the way to make child processes that run simultaneously with a single parent process.

37. Explain some of the cluster methods in NodeJS

- **Fork():** It creates a new child process from the master. The isMaster returns true if the current process is master or else false.
- **isWorker:** It returns true if the current process is a worker or else false.
- **process:** It returns the child process which is global.
- **send():** It sends a message from worker to master or vice versa.
- **kill():** It is used to kill the current worker.

38. How to manage sessions in NodeJS?

Session management can be done in NodeJS by using the express-session module. It helps in saving the data in the key-value form. In this module, the session data is not saved in the cookie itself, just the session ID.

39. Can you access DOM in Node?

No, you cannot access the DOM in NodeJS. The DOM is a browser-specific API that allows for manipulating HTML and XML documents. Since NodeJS does not run in a browser, it does not have access to the DOM.

40. How can we implement authentication and authorization in NodeJS?

Authentication is the process of verifying a user's identity while authorization is determining what actions can be performed. We use packages like Passport and JWT to implement authentication and authorization.

41. Explain the packages used for file uploading in NodeJS.the

The package used for file uploading in NodeJS is Multer. The file can be uploaded to the server using this module. There are other modules in the market but Multer is very popular when it comes to file uploading. Multer is a NodeJS middleware that is used for handling multipart/form-data, which is a mostly used library for uploading files.

42. Explain the difference between NodeJS and server-side scripting languages like Python

Features	NodeJS	Server-Side Scripting Languages (e.g., Python)
Platform	A runtime environment for executing JavaScript outside the browser.	A programming language that can be used for server-side scripting (e.g., Python, PHP, Ruby).
Language	Built on JavaScript, primarily used for asynchronous programming.	Built on Python, which is synchronous by default, but can also be used for asynchronous programming.

Features	NodeJS	Server-Side Scripting Languages (e.g., Python)
Concurrency Model	Non-blocking, event-driven I/O model using the Event Loop.	Thread-based concurrency (multi-threading) or asynchronous programming with frameworks like asyncio.
Performance	Highly performant for I/O-heavy tasks but less efficient for CPU-heavy operations due to single-threaded nature.	More suitable for CPU-heavy operations but can be less performant in handling high concurrency compared to NodeJS.
Use Case	Used for building fast, scalable, I/O-bound applications (e.g., APIs, real-time apps).	Commonly used for general-purpose programming, web development, and CPU-bound tasks (e.g., Django for web, machine learning with libraries like TensorFlow).

43. How to Connect NodeJS to a MongoDB Database?

To connect to the MongoDB database write the following code after installing the Mongoose package:

```
const mongoose = require("mongoose");

mongoose.connect("DATABASE_URL_HERE", {
   useNewUrlParser: true,
   useUnifiedTopology: true
});
```

44. How to read command line arguments in NodeJS?

Command-line arguments (CLI) are strings of text used to pass additional information to a program when an application is running

through the command line interface of an operating system. We can easily read these arguments by the global object in node i.e. process object. Below is the approach:

Step 1: Save a file as index.js and paste the below code inside the file.

```
let arguments = process.argv ;

console.log(arguments) ;
```

Step 2: Run the index.js file using the below command:

```
node index.js
```

45. Explain the NodeJS Redis module.

Redis is an Open Source store for storing data structures. It is used in multiple ways. It is used as a database, cache, and message broker. It can store data structures such as strings, hashes, sets, sorted sets, bitmaps, indexes, and streams. Redis is very useful for NodeJS developers as it reduces the cache size which makes the application more efficient. However, it is very easy to integrate Redis with NodeJS applications.

46. What is web socket?

Web Socket is a protocol that provides full-duplex (multiway) communication i.e. allows communication in both directions simultaneously. Web Socket is a modern web technology in which there is a continuous connection between the user's browser (client) and the server. In this type of communication, between the web server and the web browser, both of them can send messages to each other at any point in time.

47. Explain the util module in NodeJS

The Util module in NodeJS provides access to various utility functions. There are various utility modules available in the NodeJS module library.

- **OS Module:** Operating System-based utility modules for NodeJS are provided by the OS module.
- **Path Module:** The path module in NodeJS is used for transforming and handling various file paths.
- **DNS Module:** DNS Module enables us to use the underlying Operating System name resolution functionalities. The actual DNS lookup is also performed by the DNS Module.
- **Net Module:** Net Module in NodeJS is used for the creation of both client and server. Similar to DNS Module this module also provides an asynchronous network wrapper.

48. How to handle environment variables in NodeJS?

We use process.env to handle environment variables in NodeJS. We can specify environment configurations as well as keys in the .env file. To access the variable in the application, we use the "process.env.VARIABLE_NAME" syntax. To use it we have to install the dotenv package using the below command:

```
npm install dotenv
```

49. Explain DNS module in NodeJS

DNS is a node module used to do name resolution facility which is provided by the operating system as well as used to do an actual DNS lookup. Its main advantage is that there is no need for memorizing IP addresses – DNS servers provide a nifty solution for converting domain or subdomain names to IP addresses.

50. What is the difference between setImmediate() and setTimeout()?

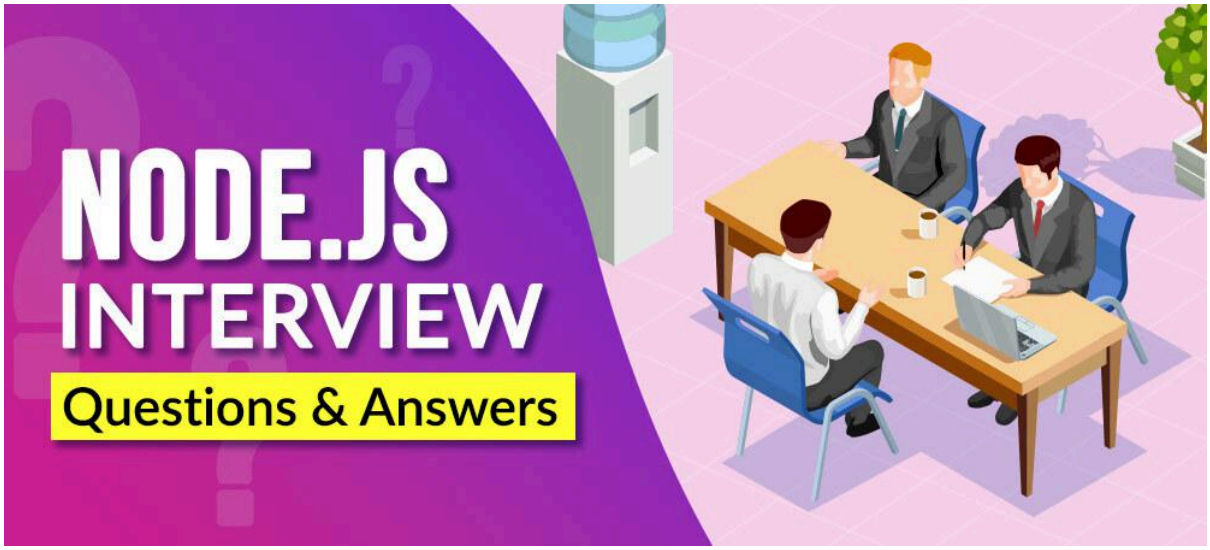
Features	steImmediate()	setTimeout()
Execution Timing	Executes the callback immediately after the current event loop phase.	Executes the callback after the specified delay (in ms).
Callback Queue	Adds the callback to the next iteration of the event loop, in the check phase.	Adds the callback to the timer queue, to be executed after the specified delay.
Timer Resolution	Has no timer; it is designed for immediate execution.	Executes only after the specified delay, which may vary slightly based on system timing resolution.
Delay	No delay, always executes after the current phase finishes.	Executes after the specified delay (minimum of 1 ms).
Use Case	Used for callbacks that need to run immediately after I/O events or timers.	Used for delaying the execution of a callback by a specific time.

51. For NodeJS, why does Google use the V8 engine?

Google **for** the V8 engine for NodeJS of the following reasons mentioned below:

- **High Performance:** V8 is a highly optimized JavaScript engine designed for speed. It compiles JavaScript directly into machine code, which makes it much faster than interpreted JavaScript.
- **Just-In-Time (JIT) Compilation:** V8 uses JIT compilation, which translates JavaScript code into machine code during execution, enabling faster execution compared to traditional interpretation.

- **Cross-platform Compatibility:** V8 is cross-platform due to which the NodeJS application can run on **that** platforms.
- **Integration with Google Chrome:** The Google Chrome by using the V8 engine in the NodeJS ensures consistency in performance and features.
- **Asynchronous I/O Efficiency: The** V8 engine can handle the non-blocking, asynchronous I/O operations which is important for handling the multiple tasks in NodeJS.



Ready to go from coding beginner to development pro? Our **DSA to Development Coding Guide** has everything you need to crush coding interviews and ace real-world projects! Limited spots available! **Enroll Now** and Transform Your Coding Skills! Also get **90% fee refund** on **completing 90%** of the course in **90 days**! Take the **Three 90 Challenge** today.

Comment
More info
Placement Training Program

Next Article

MERN Stack Interview Questions

Similar Reads

Active Directory Interview Questions - Top 50+ Questions and Answ…